

Walmart Medallion Data Pipeline

MongoDB → PostgreSQL → dbt (Bronze → Silver → Gold)

Incremental PySpark Extraction | 17 dbt Models | 3 Runners | 8 Quality Gates

Nitin — Walmart_DBT | September 2026 | v0.1.0 | github.com/Nitinx12/Walmart_DBT

Python 3.13 / 3.12 / 3.11	PostgreSQL 16	PySpark 3.5.5	dbt 1.12 / 1.11
Airflow 3.3 Celery	GX 1.21.0	Docker Compose	uv lock

One pipeline, three runners — **PowerShell, Airflow DAG, Docker image**
One warehouse — **walmart_db** with schemas **bronze / silver / gold**

This report documents the production-style medallion ETL that mirrors MongoDB operational data into a PostgreSQL warehouse and curates it through **bronze** (raw, 1:1), **silver** (deduped, typed, SCD2) and **gold** (dimensional star/snowflake) layers. The pipeline is implemented once and executed identically by three runners over eight strict stages gated by two independent test systems plus Great Expectations. It covers 9 silver and 8 gold dbt models, watermark-based incremental extraction with true upserts, containerisation with two images, CI with a live Postgres integration job, and a Streamlit+Plotly gold dashboard. All commands use **uv** and **uv.lock** is the source of truth.

Keywords — Medallion architecture, PySpark, MongoDB Spark Connector, PostgreSQL 16, dbt Core, Airflow 3.3, Great Expectations, Docker Compose, Streamlit

Contents

1	Introduction & Objectives — 3
2	Architecture at a Glance — 3
3	Technology Stack — 4
4	Repository Layout — 4
5	Pipeline — Eight Strict Stages — 5
6	Bronze Ingestion: Watermark Extraction — 6
7	Silver Layer: Clean, Typed, Historical — 7
8	Gold Layer: Dimensional Star / Snowflake — 8
9	Data Quality: Three Independent Systems — 9
10	Orchestration: One Pipeline, Three Runners — 9
11	Containerisation — 10
12	CI / CD — 10
13	BI Layer: SQL Reports & Dashboard — 11
14	Shared Utilities — 12
15	Running the Pipeline — 12
16	Observability, Security & Conventions — 13
17	Limitations & Roadmap — 13
18	Conclusion & References — 14
Appendix A Quick Reference — 14 Appendix B Environment Variables — 14	

Conventions — All file paths are relative to the repository root. Code listings show the authoritative line numbers. Stage numbers 0–7 (eight stages) match `pipeline/run_pipeline.ps1`, `main.py` and `airflow/dags/walmart_pipeline_dag.py`.

1 Introduction & Objectives

1.1 Why medallion

Operational MongoDB must stay write-optimised; analytics needs a typed, historical, dimensional warehouse. The medallion pattern isolates those concerns: **bronze** preserves source fidelity, **silver** enforces types and slowly-changing history, **gold** publishes a star schema for BI. Every hand-off between layers is a quality gate — downstream layers only build if the upstream gate passes.

1.2 Project goals

- Replicate **every** user collection from MongoDB to Postgres automatically (no hard-coded list) and keep it incrementally fresh without re-copying unchanged data.
- Cure source issues once (typing, deduplication, SCD2) so gold consumers never re-implement cleaning.
- Publish a dimensional star (**fact_order_items** + 7 dims) that powers both SQL reports (**sql/*.sql**) and an interactive dashboard.
- Gate quality twice at each hand-off: dbt built-ins (**schema.yml**) and a standalone SQL suite (**tests/bronze|silver|gold**), plus a final cross-layer Great Expectations checkpoint.
- Run the **same** logic three ways — local Windows, Airflow schedule, standalone container — without forking code.
- Ship like a team would: **uv** lockfile, Ruff + SQLFluff + pre-commit, GitHub Actions CI with a live Postgres integration job, Docker images published to GHCR.

1.3 Non-goals

Real-time streaming (batch incremental), cross-warehouse federation, and PII masking are out of scope for the current milestone.

2 Architecture at a Glance

MongoDB operational source	bronze raw, 1:1, _id PK	silver typed, deduped, SCD2	gold star / snowflake	BI reports + Streamlit
--------------------------------------	-----------------------------------	---------------------------------------	---------------------------------	----------------------------------

Fig. 1 — Medallion flow: MongoDB → bronze (PySpark watermark) → silver → gold (dbt) → BI. Postgres database is **walmart_db**; schemas default to **bronze** / **silver** / **gold** via `POSTGRES_SCHEMA_*` in `.env` (`.env.example` is the template). Control tables `etl_watermarks` and `etl_logs` live alongside bronze.

All runners set `PYTHONPATH` to the project root so `utils.*` resolves identically. The “localhost problem” is handled by rewriting `localhost` → `host.docker.internal` only in consuming layers (DAG `_PREAMBLE`, `docker run`), never by editing `.env`.

3 Technology Stack

Layer	Technology	Version / Notes
Language	Python	3.13 local, 3.12-slim Docker, 3.11 CI
Package mgr	uv	uv.lock is source of truth; never hand-edit pyproject.toml without uv lock
Compute	PySpark	3.5.5 pinned — required by mongo-spark-connector_2.12:10.4.0; 4.x is binary-incompatible
Source	MongoDB	Operational DB, auto-discovered collections
Warehouse	PostgreSQL	16 — <code>walmart_db</code> , JDBC via <code>jars/postgresql.jar</code>
Transform	dbt Core + dbt-postgres	1.12.0 / 1.11.0 — 17 models, 100+ tests
Orchestration	Apache Airflow	3.3.x, CeleryExecutor (scheduler, worker, triggerer, Redis, Postgres meta)
Quality	Great Expectations	1.21.0 — suites per layer in <code>pipeline/data_quality/suites/</code>
BI	Streamlit 1.62 + Plotly 7.0	Live gold dashboard at <code>dashboard/app.py</code>
Lint / Format	Ruff 0.16 + SQLFluff 4.2 + pre-commit	<code>ruff check/format</code> , <code>sqlfluff lint --dialect postgres</code>
Containers	Docker / Compose	<code>docker/Dockerfile</code> (standalone) + <code>docker/Dockerfile.airflow</code>
CI	GitHub Actions	<code>lint</code> → <code>dashboard-smoke</code> → <code>unit</code> → <code>DAG</code> → <code>Docker</code> → <code>integration</code> (live Postgres)

JDK is 17 (`docker/Dockerfile` and dev container) — PySpark 3.5's Mongo connector is untested on JDK 21, so the pin is intentional.

4 Repository Layout

```
airflow/dags/walmart_pipeline_dag.py | docker/Dockerfile, Dockerfile.airflow, compose.yml | docs/*.md
| jars/*.jar | pipeline/run_pipeline.ps1 + data_quality/ | scripts/python/ (extract.py, sql_test.py,
ci_seed_bronze.py, ...) | sql/ (21 files) | tests/bronze (3), silver (9), gold (7) | utils/
(engine.py, connection.py, logger.py) | dbt/models/{silver,gold} | dashboard/ (app.py, queries.py,
theme.py) | reports/ | main.py
```

Conventions: **snake_case**, type hints on every signature, one function = one responsibility (see `scripts/python/extract.py` stage split), short “why not what” comments, **uv run ruff format/check** before commit.

5 Pipeline — Eight Strict Stages

Fixed order. If stage order or the definition of “success” changes, **pipeline/run_pipeline.ps1** and **airflow/dags/walmart_pipeline_dag.py** must change together (convention, `AGENTS.md`: Pipeline Stage Rule).

#	Stage	Runner command	Gate
0	Preflight	<code>uv run python -c "import pyspark" + prefix check 3.5.x</code>	Mismatch → fail fast; container refuses to self-heal

#	Stage	Runner command	Gate
1	Extract	uv run python scripts/python/extract.py	Per-collection VALIDATION FAILED → non-zero exit
2	Bronze SQL tests	uv run python scripts/python/sql_test.py tests/bronze	3 checks; violating rows → FAIL
3	dbt silver	cd dbt && dbt run --select silver; dbt test --select silver	schema.yml tests; failure stops stage
4	Silver SQL tests	.../sql_test.py tests/silver	9 checks
5	dbt gold	dbt run --select gold + test	8 models (7 dims + 1 fact)
6	Gold SQL tests	.../sql_test.py tests/gold	7 checks
7	Great Expectations	python -m pipeline.data_quality.run --layer all	Bronze/Silver/Gold suites; exit 1=data fail, 2=build error

Stop-on-first-failure: **run_pipeline.ps1:99** (Stop-Pipeline) and **main.py:244** (raise) halt immediately; Airflow's default **all_success** trigger does the same — downstream tasks go to **upstream_failed**. Logs go to **logs/pipeline_YYYY-MM-DD.log** (utils/logger.py) plus **{schema}.etl_logs** audit rows. Header "STEP n / 7" plus stage 0 = 8 stages stays honest via **main.py:37 TOTAL_STEPS = 7** assert.

6 Bronze Ingestion: Watermark Extraction

File: **scripts/python/extract.py** (~1230 lines) — the only place that touches MongoDB.

6.1 Auto-discovery

discover_collections() lists **mongo_db.list_collection_names()** and drops names starting with **system**. (extract.py:127). No collection list is hard-coded; **--tables** is an optional filter for ad-hoc runs.

6.2 Watermark column priority

Per collection, a single document sample is inspected; first present candidate wins: **updated_timestamp > updated_at > created_timestamp > created_at** (extract.py:151). **updated_*** is preferred because documents mutated after insert (e.g. order status → Cancelled) must be re-synced.

6.3 Incremental pushdown

When a watermark column and prior **last_watermark_value** exist, the Spark read is filtered server-side: **watermark_str = since.strftime("%Y-%m-%d %H:%M:%S")** then **json.dumps([{"\$match": {"col": {"\$gt": watermark_str}}])** via **reader.option("aggregation.pipeline", ...)** (extract.py:656). Comparison is string-to-string because the source stores timestamps as "YYYY-MM-DD HH:MM:SS" strings, not BSON dates — MongoDB never matches across BSON types in \$gt.

6.4 Upsert, not append

_id (cast to string, **sanitize_for_postgres:672**) is the merge key. Existing tables get a scratch staging table **_stg_{collection}** written with **mode("overwrite")**, then a single **INSERT ... ON CONFLICT (_id) DO UPDATE** with **(xmax = 0) AS inserted** to split counts (extract.py:721). A unique index on **_id** is ensured first (**ensure_unique_id_index:409**); creation failure (duplicate **_id** from a prior append-only run) falls back to plain append with a warning.

6.5 Nested fields

StructType / ArrayType / MapType columns are serialised to JSON strings via **to_json()** (extract.py:680) — the only lossy step, required because plain JDBC has no native struct type. Flattened field names are reported in the Rich final panel.

6.6 Control tables

- **{bronze}.etl_watermarks** — one row per collection: `incremental_column`, `last_watermark_value`, `last_run_mode`, `rows_inserted/updated`, `updated_at`. Upserted via `ON CONFLICT (table_name)` (extract.py:528).
- **{bronze}.etl_logs** — one audit row per collection per run: `mode`, `counts before/after`, `validation status`, `error_full` `traceback`, `duration`. Best-effort, never fails the run.
- **ensure_control_tables:448** is idempotent and forward-migrates columns added in later versions.

6.7 Validation & reporting

After each merge, **validate_collection:619** recounts the target and expects **before + inserted**. A Rich report prints per-collection rows, watermark state, flattened fields, and failures (render_report:916); same data lands in the daily log file and `etl_logs`. CLI: `--tables`, `--full-refresh`, `--dry-run`, `--watermark-column` (extract.py:1117).

7 Silver Layer: Clean, Typed, Historical

dbt/dbt_project.yml:35 materialises all silver models as **table** in schema **silver**. Nine models mirror bronze 1:1 but add typing, dedup, and canonicalisation.

Model	Key cleaning	Notable logic
silver.brands	dedup on <code>_id</code> , trim/case	Coalesce brand name variants
silver.categories	dedup, null handling	Canonical category list
silver.customers	dedup, email/phone	<code>is_active</code> flag
silver.employees	dedup, date parsing	Joins to stores
silver.orders	timestamp parsing, status	<code>order_timestamp</code> typed
silver.order_items	<code>line_amount = qty*price</code>	FK to orders + products
silver.payment_methods	dedup, type norm	Lookup for gold dim
silver.products	dedup, price typing	Links brands + categories
silver.stores	dedup, geo	Store master

Each model has a **schema.yml** with **not_null**, **unique**, **relationships**, **accepted_values** tests. Grain is one row per source document — no aggregation yet. **tests/silver/** adds 9 PL/pgSQL checks (row counts, orphan FKs, distinctness) via **scripts/python/sql_test.py**.

8 Gold Layer: Dimensional Star / Snowflake

dbt/dbt_project.yml:39 materialises gold as **table** in schema **gold**. Eight models:

Model	Type	Grain
gold.dim_stores	Dimension	One row per store
gold.dim_customers	Dimension	One row per customer
gold.dim_products	Dimension (snowflake)	One row per product
gold.dim_brands	Sub-dimension	One row per brand
gold.dim_categories	Sub-dimension	One row per category
gold.dim_orders	Dimension	One row per order header
gold.dim_payment_methods	Dimension	One row per payment method
gold.fact_order_items	Fact (incremental merge)	One row per order line; FKs to <code>dim_orders</code> , <code>dim_products</code>

- **fact_order_items.sql:1** is the only incremental gold model — **unique_key=order_item_id**, **incremental_strategy='merge'**, 3-day lookback (**updated_timestamp >= MAX(updated_timestamp) - 3 days**) to catch late-arriving updates.

- Dimensions are full-refresh tables rebuilt from silver each run — cheap at this scale and avoids SCD bookkeeping where not needed.
- **gold_loaded_at = CURRENT_TIMESTAMP** stamps lineage.
- The star is snowflaked only at products → brands/categories — intentional, because brand/category reports aggregate directly on those sub-dims.

Gold is gated by **dbt test --select gold** plus 7 standalone checks under **tests/gold/**.

9 Data Quality: Three Independent Systems

System	Location	Runner	Semantics
dbt tests	dbt/models/*/schema.yml	dbt test --select silver gold	Column/table contracts (null, unique, FK, accepted values)
SQL suite	tests/{bronze,silver,gold}/*.sql	sql_test.py tests/	SELECT returning violating rows = FAIL; DO \$\$ RAISE EXCEPTION = FAIL; one conn/layer, rollback on fail
Great Expectations	pipeline/data_quality/suites/	python -m pipeline.data_quality.run --layer all	One checkpoint per layer ({layer}_checkpoint), Fluent API 1.x; exit 1=data fail, 2=build error

The SQL suite auto-detects mode per file via **psycopg2 ResourceClosedError** (tests/unit/test_sql_test.py:89). GX suites are built **add_or_update** so re-runs reconcile instead of duplicating, and all requested layers run even if an earlier one fails unless **--fail-fast** is passed (pipeline/data_quality/run.py:150). Together: bronze 3 checks, silver 9, gold 7, plus ~23 GX suites and 100+ dbt tests.

10 Orchestration: One Pipeline, Three Runners

Runner	Entry point	How it loads env & runs
Windows host	pipeline/run_pipeline.ps1 + run_pipeline.bat	Import-DotEnv overwrites process env; Assert-Prerequisites checks 7 paths; Invoke-PipelineCommand shells out via uv
Standalone container	main.py (docker/Dockerfile ENTRYPOINT)	load_dotenv via setdefault (container env wins); in_container() refuses to self-heal PySpark inside image
Airflow DAG	airflow/dags/walmart_pipeline_dag.py dag_id=walmart_medallion_pipeline	Every @task.bash prefixes _PREAMBLE: source .env, rewrite localhost→host.docker.internal, force PYSPARK_PYTHON=/app/.venv/bin/python, set DBT_PROFILES_DIR=/app/docker/dbt

DAG tasks form a linear chain **preflight** → **extract** → **bronze_tests** → **silver_run** → **silver_test** → **silver_sql** → **gold_run** → **gold_test** → **gold_sql** → **gx** with default **retries=0**, **max_active_runs=1**. Fail-fast is free from Airflow's **all_success**. **_PREAMBLE** handles the “localhost problem” (docs/architecture.md:29): .env keeps localhost for local uv run; only consuming layers rewrite.

11 Containerisation

Two images, two jobs (docs/docker.md):

- **docker/Dockerfile** — standalone runner: **python:3.12-slim + JDK 17 + uv**, copies **jars/**, installs from **uv.lock**, sets **RUNNING_IN_DOCKER=1**, **ENTRYPOINT ["python", "main.py"]**.
- **docker/Dockerfile.airflow** — CeleryExecutor base: **apache/airflow:3.3.0 + JDK 17 + uv**, same **jars/** layer, used by **docker/compose.yml** for 6 services (apiserver, scheduler, dag-processor, worker, triggerer, postgres+redis).
- **docker/compose.yml** mounts the project root to **/app** and shadows **/app/.venv** with an anonymous volume so the container's Linux venv never clobbers the host's Windows venv. **airflow-worker** gets **extra_hosts: host.docker.internal:host-gateway**.

- Dev container (`.devcontainer/devcontainer.json`) mirrors the standalone Dockerfile (JDK 17, uv preinstalled) so PySpark works without local Java.

12 CI / CD

`.github/workflows/ci.yml` — 6 jobs, PYTHON_VERSION 3.11, AIRFLOW_VERSION 3.3.0:

1. **lint** — `ruff check, ruff format --check, sqlfluff lint dbt/models sql/ tests/ --dialect postgres`
2. **dashboard-smoke** — installs *only* `dashboard/requirements.txt` (no databricks-sql-connector), asserts databricks not importable, then imports `dashboard.queries` with dummy env — catches the unconditional-databricks regression that broke Streamlit Cloud.
3. **unit-tests** — `pytest tests/unit -v --cov=utils --cov=scripts` (28 tests, mocked, no live DB)
4. **dag-integrity** — installs Airflow via constraints file and asserts `DagBag` loads `walmart_medallion_pipeline` with no import errors or cycles.
5. **docker-build** — builds both `docker/Dockerfile` and `docker/Dockerfile.airflow` with Buildx cache, `ifs: true` so `jars/*.jar` are real files.
6. **integration** — live Postgres 16 service; `ci_seed_bronze.py` seeds bronze from fixtures, then the exact gate sequence: bronze SQL → `dbt run/test silver` → silver SQL → `dbt run/test gold` → gold SQL → GX `--layer all`. Uses absolute `DBT_PROFILES_DIR` and `PYTHONPATH` fixes documented inline.

Images are published to GHCR on merge/tag; releases are cut via `CHANGELOG.md`.

13 BI Layer: SQL Reports & Dashboard

13.1 Hand-written SQL (sql/, 21 files)

`00_init_schema.sql` creates schemas and extensions; `01_brands.sql` .. `03_payment_methods.sql` and `08_order_status_analysis.sql` .. `20_no_sales_date_analysis.sql` are analytical queries (brand/category revenue share, payment mix, ranking, Pareto, market basket, ABC, cohort) and two PL/pgSQL functions (`05_fn_product_report.sql`, `06_fn_sales_trend.sql`) plus a trigger (`07_trg_loaded.sql`). They query gold directly and are the ancestors of `reports/brand_report.md` and `category_report.md`.

13.2 Gold dashboard (dashboard/)

`dashboard/app.py` (766 lines) is a Streamlit app with Plotly charts. All SQL lives in `queries.py` (parameterised, aggregated in Postgres, cached via `st.cache_data`); styling in `theme.py`; cards/chips in `components.py`. Filters (date presets 7D/30D/90D/YTD/All, stores, categories, order statuses, granularity Day/Week/Month) build an immutable `Filters` key; KPIs show period-over-period deltas; tabs are Overview / Trends & Mix / Breakdown / Data. Graceful empty states when gold is unpopulated. Runs with `uv run streamlit run dashboard/app.py` or on Streamlit Cloud with `st.secrets` fallback (`app.py:34`).

13.3 Existing reports (reports/)

`reports/brand_report.md` (7 brands, \$18.9M total, Apple 15.75% leader) and `reports/category_report.md` (Electronics 17.76% leader, Home 14.34% outlier) are the reference BI outputs — both derived from gold CTEs that `LEFT JOIN dim_* → fact_order_items` so zero-sales entities remain visible.

14 Shared Utilities

`utils/engine.py` — Loads `.env` at import, validates required vars (`POSTGRES_*`, `MONGO_*`) with `OSError` on missing, warns on optional `POSTGRES_SCHEMA_*` and Databricks vars, casts `POSTGRES_PORT` to int. Hard-fails fast so every script gets a clear error.

`utils/connection.py` — Cached factories: `get_postgres_engine()` (SQLAlchemy, SSL params), `get_mongo_db()` (pymongo with ping), `get_databricks_connection()` (lazy databricks_sql_connector, `ModuleNotFoundError` if extra not installed — dashboard-safe).

utils/logger.py — **get_logger(name)** returns a logger with console + daily file handlers (**logs/pipeline_YYYY-MM-DD.log**), **propagate=False**, configurable levels. Used by extract, dashboard, and GX runner.

Patching for tests is via **tests/unit/test_*.py** with **unittest.mock** and small fixtures — never real DB data.

15 Running the Pipeline

Local (Windows) — mirrors README “Run it”:

```
uv venv
.venv\Scripts\activate # Windows; source .venv/bin/activate on Linux/Mac
uv sync # from uv.lock
cp .env.example .env # fill POSTGRES_*, MONGO_*, optional DATABRICKS_*

./pipeline/run_pipeline.ps1 # 8 stages
# or: uv run python main.py
# or: uv run python scripts/python/extract.py --dry-run --tables orders,customers

uv run python scripts/python/sql_test.py tests/bronze
uv run dbt run --select silver --project-dir dbt && uv run dbt test --select silver --project-dir dbt
uv run python -m pipeline.data_quality.run --layer all

uv run streamlit run dashboard/app.py

uv run ruff format . && uv run ruff check . && uv run sqlfluff lint dbt/models --dialect postgres
uv run pre-commit run --all-files
uv run pytest tests/unit -v --cov=utils --cov=scripts
```

Docker & Airflow:

```
docker build -f docker/Dockerfile -t walmart-pipeline .
docker run --env-file .env walmart-pipeline

docker compose -f docker/compose.yml up
# UI at http://localhost:8080 (airflow/airflow), DAG: walmart_medallion_pipeline
```

Health and security scripts: **uv run python scripts/python/health_check.py** and **security_check.py** (see [docs/health.md](#)).

16 Observability, Security & Conventions

- Git workflow: one file/logical change per commit, **git add <file>** + prefix **extract:** / **dbt:** / **airflow:** / **docker:** / **tests:** / **utils:** / **docs:** (AGENTS.md, docs/git-workflow.md), rebase before push, no **.venv** / **__pycache__** / **.env** / **data/raw|processed** committed.
- Secrets: **.env** is git-ignored; **.env.example** is the template; **detect-private-key** pre-commit hook; Databricks token never baked into images.
- Logging: Rich console + structured file logs + **etl_logs** table — three places to diagnose a failed collection without re-running.
- Idempotency: control tables and watermark upserts are **CREATE IF NOT EXISTS / ON CONFLICT** — re-running never duplicates state.
- Offline Spark: **jars/postgresql.jar** is vendored; the 5 Mongo connector jars are vendored when present in **jars/** — otherwise resolved once via Maven and a tip is printed (extract.py:304).

17 Limitations & Roadmap

Area	Next step
Watermark type	Source timestamps are strings; migrating Mongo to BSON dates would allow native date pushdown and remove the string-format coupling.
SCD2	Silver history is dedup-only today; true SCD2 snapshots (dbt snapshots on dim_customers / dim_stores) are sketched in docs/dbt.md but not yet materialised.

Area	Next step
Gold incremental	Only fact_order_items is incremental; dimensions are full refresh — fine at current scale, revisit if counts grow 10x.
Streaming	Batch-only; CDC via Debezium or Mongo change streams would enable near-real-time bronze.
PII / masking	No column-level masking in silver; add views or dbt post-hooks before exposing gold externally.
Data Docs	GX checkpoints have no UpdateDataDocsAction yet — add once gx/uncommitted is published.
Tests	Bronze schema.yml tests are lighter than silver/gold — expand with dbt_expectations for cross-column rules.

18 Conclusion

The Walmart medallion pipeline delivers a clean separation between operational and analytical concerns without duplicating logic across execution environments. Auto-discovery plus watermark upserts keep bronze fresh and auditable; dbt turns that bronze into a typed silver and a dimensional gold that is directly queryable for reports and a live dashboard; two independent test systems plus Great Expectations gate every hand-off; and a single stage definition runs identically on a developer laptop, in Docker, and on a schedule in Airflow. The codebase is small enough to read in an afternoon (**extract.py** + 17 models + **sql_test.py** + **run.py**) yet strict enough to catch schema drift, load errors, and data regressions before they reach gold.

References

- Project repository — github.com/Nitinx12/Walmart_DBT
- Live dashboard — Streamlit Cloud
- dbt documentation — docs.getdbt.com
- Great Expectations 1.x Fluent API — docs.greatexpectations.io
- MongoDB Spark Connector — mongodb.com/docs/spark-connector
- Apache Airflow 3.3 — airflow.apache.org

Appendix A Quick Reference

Task	Command
Create env	uv venv
Install deps	uv sync
Add dep	uv add
Run extract	uv run python scripts/python/extract.py
Run dbt tests	uv run dbt test --select silver
Run SQL suite	uv run python scripts/python/sql_test.py tests/
Run GX	uv run python -m pipeline.data_quality.run --layer all
Format	uv run ruff format .
Lint	uv run ruff check .
Stage one file	git add
Commit	git commit -m "": "

Appendix B Environment Variables (.env.example)

```
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DATABASE=walmart_db
POSTGRES_USERNAME=postgres
POSTGRES_PASSWORD=changeme
POSTGRES_SCHEMA_BRONZE=bronze
POSTGRES_SCHEMA_SILVER=silver
POSTGRES_SCHEMA_GOLD=gold
MONGO_URI=mongodb://localhost:27017
MONGO_DB=walmart
PYSPARK_PYTHON=python
PYSPARK_DRIVER_PYTHON=python
DATABRICKS_HOST=
DATABRICKS_HTTP_PATH=
DATABRICKS_TOKEN=
```

Generated from repository state at `main@aa3d139` — September 2026. For the full system design see `docs/architecture.md` and `README.md`. LaTeX source is `reports/report.tex`; this PDF was built via `scripts/python/build_report.py` (reportlab) so it renders even without a local TeX engine.